

Операционные системы

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Загидуллина Элина Динаровна

2026-04-29

Цели и задачи работы

Процесс выполнения лабораторной работы

Выводы по проделанной работе

1. Цели и задачи работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

1 Выполнить 4 задания

2. Процесс выполнения лабораторной работы

1. Используя команды `getopts` `grep` напишем командный файл, который анализирует командную строку с ключами и выполним его: `-i inputfile` — прочитать данные из указанного файла; `-o outputfile` — вывести данные в указанный файл; `-p шаблон` — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк;

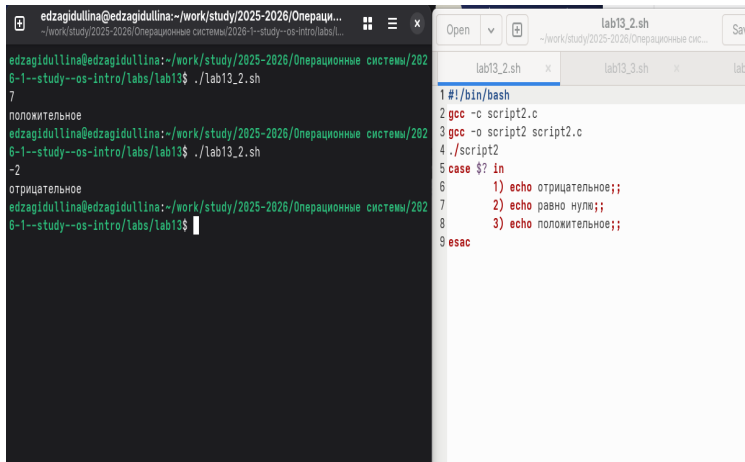
а затем ищет в указанном файле нужные строки

Выполнение работы

```
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_1.sh -i text.txt -o fout.txt -p 16
16:файлы, проецируемые в память.
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ cat fout.txt
1 #!/bin/bash
2 cflag=0;
3 nflag=0;
4 while getopts i:o:p:C:n opt
5 do
6 case $opt in
7 i) ival=$OPTARG;;
8 o) oval=$OPTARG;;
9 p) pval=$OPTARG;;
10 C) cflag=1;;
11 n) nflag=1;;
12 esac
13 done
14 if [ $cflag -a $nflag ]
15 then
16 grep -n $pval $ival>$oval
17 elif test $cflag
18 then
19 grep $pval $ival>$oval
20 elif test $nflag
21 then
22 grep -n -i $pval $ival>$oval
23 else
24 grep -i $pval $ival>$oval
25 fi
```

Рисунок 1: Задание 1

2. Напишем сначала на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем завершим программу при помощи функции `exit(n)`, передавая информацию о коде завершения в оболочку. Командный файл вызовет эту программу и, проанализировав с помощью команды `$?`, выдаст сообщение о том, какое число было введено



The image shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of a script named `lab13_2.sh`. The script's output is as follows:

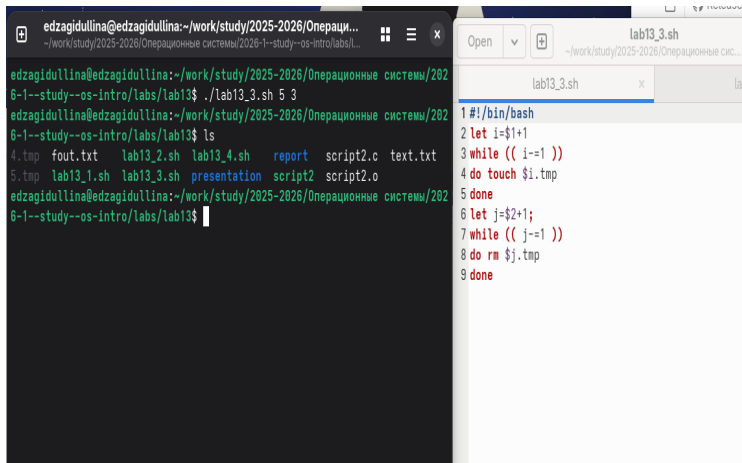
```
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_2.sh
7
положительное
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_2.sh
-2
отрицательное
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$
```

The code editor on the right shows the content of the `lab13_2.sh` script:

```
1#!/bin/bash
2gcc -c script2.c
3gcc -o script2 script2.c
4./script2
5case $? in
6    1) echo отрицательное;;
7    2) echo равно нулю;;
8    3) echo положительное;;
9esac
```

Рисунок 2: Задание 2

3. Напишем командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N



The image shows a terminal window on the left and a file editor on the right. The terminal window displays the execution of a script named `lab13_3.sh` with arguments `5 3`. The user runs `ls` and lists the contents of the directory, which include `4.tmp`, `fout.txt`, `lab13_2.sh`, `lab13_4.sh`, `report`, `script2.c`, and `text.txt`. The file editor on the right shows the content of `lab13_3.sh`, which is a shell script that increments a counter `i` and `j` and creates files `i.tmp` and `j.tmp`.

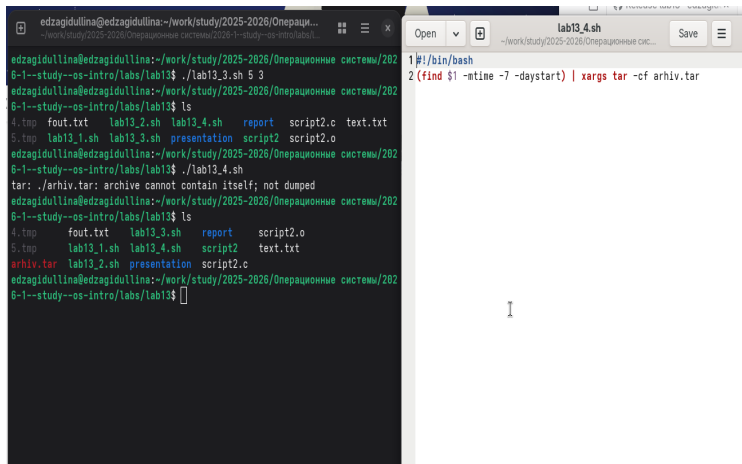
```
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_3.sh 5 3
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ls
4.tmp  fout.txt  lab13_2.sh  lab13_4.sh  report  script2.c  text.txt
5.tmp  lab13_1.sh  lab13_3.sh  presentation  script2  script2.o
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$
```

```
#!/bin/bash
1 let i=$1+1
2 while (( i-=1 ))
3 do touch $i.tmp
4 done
5 let j=$2+1;
6 while (( j-=1 ))
7 do rm $j.tmp
8 done
```

Рисунок 3: Задание 3

4. Напишем командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицируем его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад.

Выполнение работы



The image shows a terminal window on the left and a file editor on the right. The terminal window displays the following commands and output:

```
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_3.sh 5 3
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ls
4.tmp  fout.txt  lab13_2.sh  lab13_4.sh  report  script2.c  text.txt
5.tmp  lab13_1.sh  lab13_3.sh  presentation  script2  script2.o
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ./lab13_4.sh
tar: ./arhiv.tar: archive cannot contain itself; not dumped
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$ ls
4.tmp  fout.txt  lab13_3.sh  report  script2.o
5.tmp  lab13_1.sh  lab13_4.sh  script2  text.txt
arhiv.tar  lab13_2.sh  presentation  script2.c
edzagidullina@edzagidullina:~/work/study/2025-2026/Операционные системы/2026-1--study--os-intro/labs/lab13$
```

The file editor on the right shows the contents of the `lab13_4.sh` script:

```
1 #!/bin/bash
2 (find $1 -mtime -7 -daystart) | xargs tar -cf arhiv.tar
```

Рисунок 4: Задание 4

3. Выводы по проделанной работе

В данной работе мы изучили основы программирования в оболочке ОС UNIX и писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.